

Week 02 Explained Student Worksheet

Regular Expressions for Text Processing

Name _____ Student ID _____ Group _____

Extract useful values from noisy text before cleaning it. Use the supplied sales paragraph to practice the re module and distinguish extraction from replacement.

Use this worksheet during the two-hour practical session. Read, predict, run, and explain. The existing lab sheet contains the complete program; this companion explains key ideas and provides space to record your own evidence. Python 3.10 or later is sufficient; no pip packages are needed.

What you should be able to do

- Write raw-string regex patterns for digits, words, and whitespace.
- Extract numbers and email-like strings with `re.findall()`.
- Remove symbols and normalize whitespace with `re.sub()`.
- Compare character classes and repetition operators on short inputs.

Understand the concepts

A regular expression describes a pattern of characters. `re.findall(pattern, text)` returns non-overlapping matches. `re.sub(pattern, replacement, text)` returns a modified string. Neither function changes the original string in place.

Use raw strings such as `r"\d+"` to preserve backslashes for the regex engine. A normal string `"\d+"` also works. Some unrecognized Python escapes may currently survive with warnings; a recognized escape such as `"\b"` becomes a backspace. The reliable habit is to use raw patterns.

The slide pattern `r"\S+@\S+"` extracts email-like spans. It is deliberately simple and can include trailing punctuation. Extract emails before removing symbols: deleting `@` and dots destroys the address structure. This exercise is text extraction, not comprehensive email validation.

The slide cleaning pipeline deletes characters outside ASCII letters, digits, and whitespace, then collapses whitespace. Deleting punctuation can join text: `data-base` becomes `database` and email addresses lose punctuation. Week 3 instead extracts letter sequences, which preserves boundaries between separated words.

By default in Python str patterns, `\d` and `\w` are Unicode-aware; `\w` includes letters, digits, and underscore. `[a-zA-Z]` selects ASCII letters explicitly. A dot matches any character except newline unless `DOTALL` is enabled.

Predict before running

For Order 1234 costs \$56, predict the lists returned by the patterns `r"\d"` and `r"\d+"`.

Prepare your terminal

Extract the course and open a terminal inside Week_02_Regex_Text_Processing. Keep docs and data beside the Python files. The main demonstration is lab02_regex.py; your exercise starter is exercises_lab02.py. Starters run but deliberately contain unfinished TODO functions.

```
py -3.13 lab02_regex.py
py -3.13 exercises_lab02.py
```

Use py -3 or python instead if that is your working Python 3.10+ command.

Worked example with explanation

This complete example shows the two-stage cleaning pattern from the slides. Extraction happens first, while punctuation and email separators still exist.

```
import re
text = "Hello!! My age is 25@@ years old###"
print(re.findall(r"\d+", text))
text = re.sub(r"[^a-zA-Z0-9\s]", "", text)
text = re.sub(r"\s+", " ", text).strip()
print(text)
```

Expected output from an executed example

```
['25']
Hello My age is 25 years old
```

How each stage produces the result

1. The raw pattern `r"\d+"` asks for one or more consecutive digits. `findall` returns matched strings, so the age is "25", not the integer 25.
2. Inside `[^a-zA-Z0-9\s]`, the caret negates the character class. The first substitution removes characters that are neither ASCII letters, digits, nor whitespace.
3. The second substitution replaces each run of whitespace with one ordinary space. It collapses tabs and newlines too. `strip` then removes leading and trailing spaces.
4. Extract values from the original text before cleaning. Removing `@` and dots destroys address structure. Deleting punctuation can also merge words: `data-base` becomes `database`.

Explain one intermediate value in your own words

Read the weekly program

The following excerpt is copied exactly from lab02_regex.py. It depends on the imports, constants, and helper functions in that file. Read it there rather than pasting it into an empty script and expecting the excerpt to run alone.

```
def clean_text(text):
    no_symbols = re.sub(r"^[a-zA-Z0-9\s]", "", text)
    return re.sub(r"\s+", " ", no_symbols).strip()
```

Code walkthrough

1. `clean_text` accepts one string and returns a new string. Strings are immutable; the original input remains available for separate extraction.
2. `no_symbols` stores the first transformation. Naming this intermediate result makes the two operations easier to inspect and debug.
3. The pattern `r"\s+"` has a different purpose from `r"\r\n?"`. The latter can normalize Windows CRLF and lone CR endings to LF while retaining line boundaries.
4. The demonstration pattern `r"\S+@\S+"` finds email-like spans, not validated email addresses, and may capture trailing punctuation. Also, Python string patterns make `\d` and `\w` Unicode-aware; `\w` is not letters only.

If the first substitution uses a space instead of an empty string, how does the result for data-base change?

Connection to the course

Week 3 uses regex to extract token boundaries rather than delete every symbol. Comparing these approaches explains why preprocessing choices change the vocabulary.

Guided practical activity

Work through the steps in order. Compare your predictions with observations and record at least one specific value per experiment. Keep original baseline data until you have saved the first transcript. Put experiments that change files in a disposable copy.

Step 1 Open `data/paragraph.txt` and read the full four-line sales paragraph. Keep the original text for extraction.

Keep a variable holding the untouched paragraph. It is the source for all extraction experiments.

Step 2 Extract all numbers using `r"\d+"` and all email-like spans using `r"\S+@\S+"`.

Inspect each returned list and check punctuation around the addresses. Treat a match as a candidate rather than proof of a valid address.

Step 3 Remove symbols using `r"[^a-zA-Z0-9\s]"` and an empty replacement.

Print the intermediate cleaned string before collapsing whitespace. Observe exactly which characters disappeared.

Step 4 Collapse repeated spaces, tabs, and newlines using `r"\s+"`. Strip outer spaces and print the cleaned paragraph.

Compare `repr` before and after replacement so that tabs and newline characters are visible.

Step 5 Compare `r"\w+"` with `r"[a-zA-Z]+"` on `User_7`. Test each quantifier on `data/pattern_practice.txt`.

`User_7` distinguishes word characters from ASCII letters. Predict before executing, and explain the role of the underscore and digit.

Step 6 Finish the starter, returning numbers, emails, and cleaned text. Check all three results independently.

Complete the starter functions independently. Preserve separate results for extraction and cleaning rather than replacing the original paragraph.

Test record

Check to demonstrate	Actual observation	Pass or revise
Digit sequences are extracted from the unchanged input		
An address is extracted before @ is removed		
Whitespace collapse is distinguished from newline normalization		

Record one prediction that changed after running the program and explain why

Independent work and submission

Complete these selected tasks in your own exercise file. Use the existing lab sheet for additional exercises. Do not submit the unchanged starter or simply import a solution in place of implementing the requested function.

- Basic: compare `r"\d"` and `r"\d+"` on Order 1234 costs \$56. Explain why the lists differ.
- Implementation: extract words using `r"[a-zA-Z]+"` and email-like strings from the supplied paragraph.
- Implementation: remove punctuation `[!?,.]` from a short sentence, then remove all unwanted symbols from the sales paragraph.
- Debugging: explain what is wrong with `r"[a-z | A-Z]"` when the intention is to match letters. Correct it and add `+` to extract whole sequences.

Diagnose common errors

The pattern matches spaces or a vertical bar. Inside a character class, `[a-z | A-Z]` includes literal spaces and `|`. Use `[a-zA-Z]+` for complete ASCII-letter sequences.

No address remains after cleaning. Extract from the original input. Destructive substitution removes characters that the address pattern depends on.

Submit these items

- A completed starter and the unchanged supplied paragraph.
- Outputs for numbers, emails, and the cleaned paragraph, plus your newline-normalization experiment.
- A short explanation of why extraction must precede destructive cleaning.

Your evidence should identify the input or query, expected result, actual result, and explanation. State any document edits or preprocessing changes so the result can be reproduced. Include this completed worksheet or a clearly numbered digital response.

Exit questions

How do the return values of `findall` and `sub` differ?

Why are raw strings useful in regex patterns?

Optional challenge

Write a cleaning function with a `keep_newlines` option. Preserve LF line boundaries while collapsing repeated spaces and tabs, then test it on mixed Windows and Unix line endings.

Source for this companion

Based on the supplied Week 02 lab sheet, shipped program, exercise starter, and sample data. The complete main program and `expected_output.txt` in this folder are the references for checking your baseline run.